Generator

Friday, September 19, 2025 12:02 AM

Iterator:

'''

- it is a phenomenon of a Traversing through the sequence to get the value one by one and the function
- which is used for iteration is called 'Iterator'

```
for i in [10,20,30,40]:
    print(i)
```

1.iter():it is a inbuilt function which is used to point to the intial address of the collection

2.next():it is a inbuilt function which is used to hold the value or data

```
l=[10,20,30,40]
i=iter(l)
print(next(i))
print(next(i))
print(next(i))
print(next(i))
print(next(i))#stop iteration error
```

'''

- **Definition**
  - An **iterator** is an object that allows sequential access to elements of a collection, one element at a time.
- **Two main methods**
  - __iter__() → returns the iterator object itself.
  - __next__() → returns the next element, raises StopIteration when no elements are left.
- **Iterable vs Iterator**
  - **Iterable**: An object that can return an iterator (e.g., list, tuple, dict, set, string).
  - **Iterator**: The object that actually does the iteration, produced by calling iter(iterable).

```
nums = [1, 2, 3]
it = iter(nums)  # creating iterator
print(next(it))  # 1
print(next(it))  # 2
print(next(it))  # 3
```

- **One-time use**
  - Iterators are **exhausted** after traversing once.
  - You cannot reset an iterator; you need to create a new one.
- **Lazy evaluation**
  - Iterators produce values **on demand** instead of storing all elements in memory.
  - This makes them memory efficient for large data.
- **StopIteration exception**

- When no items remain, calling next() raises StopIteration.
- for loops handle this automatically behind the scenes.

➢ **For loop uses iterator internally**
- A for loop automatically calls iter() and next() on an iterable.
- That's why you can loop over lists, tuples, dicts, etc.

Genrator:

➢ generator is a process of creating new collection using yield keyword where the function is used to perform

➢ geration or which is used for generation process id called 'Generator'

```
#only using return
def sample():
    print('hello')
    return 1
    print('hai')
    return 2
    print('bye')
    return 3
print(sample())
```

- As soon as Python hits the first return, the function **exits immediately**.
- Any code after return is **unreachable** (never executes).
- Output:

```
hello
1
```

```
#only using yield
def sample():
    print('hello')
    yield 1
    print('hai')
    yield 2
    print('bye')
    yield 3
print(list(sample()))
```

- yield makes the function a **generator**.
- Instead of exiting, it **pauses** the function and remembers the state.
- Each call to next() resumes from where it left off.
- list(sample()) forces the generator to run until exhaustion.

- Output:

```
hello
hai
bye
[1, 2, 3]
```

```
#Combination of yield and return
def sample():
    print('hello')
    yield 1
    print('hai')
    yield 2
    print('bye')
    return 3
print(set(sample()))
```

- A generator function can also use return, but here return means **StopIteration** is raised.
- The value after return becomes the **value of StopIteration**, not part of the yielded sequence.
- Since we use set(sample()), only the yield values {1, 2} are collected.
- The final return 3 is **not included** in the output set.
- Output:

```
hello
hai
bye
{1, 2}
```

return:

- return is used to stop the function execution by returning the value and it will not go back to the same function for further instruction execution.
- if we use return in a function it is normal function.
- normal function can be called directly

yield:

- is is going pause the execution to store the value again it will come back to the same program further
- instruction execution.
- when we use yield inside the function it is called as 'Gerator function'.
- generator function we have to go with type casting until it contain object address

Q.wap program to create square of all the integers between 1 to 10

```
def sqr():
```

```
    out=[]
    for i in range(1,11):
        out.append(i**2)
    return out
print(sqr())
#by using Generator
def gen_sqr():
    for i in range(1,11):
        yield i**2
print(list(gen_sqr()))
```

2.wap to extract all the strings from list collection only if it pallindrome

```
#Generator Function

def ex(l):
    for i in l:
        if type(i)==str and i==i[::-1]:
            yield i
print(tuple(ex(['nayan','hai',10,2.5,10+5j,'aba'])))
```

wap to extract all the prime numbers between the range m,n

```
def is_prime(n):
    for i in range(2,n):
        if n%i==0:
            return False
        return True

def prime(m,n):
    for i in range(m,n+1):
        if  is_prime(i)==True:
            yield i
print(list(prime(2,50)))
```

wap to get the following output

```
#s='GeneRator@123'
#output={'G':'g','e':'E','n':'N'.....'@':'@','1':'1'....}
def sample(s):
    for i in s:
        if 'A'<=i<='Z':
            yield i,chr(ord(i)+32)
        elif 'a'<=i<='z':
            yield i,chr(ord(i)-32)
```

```
            else:
                yield i,i
        print(dict(sample('GeneRator@123')))
```

wap to create the following dictionary using generator

```
#out={'A':65,'B':66,'C':67,'D':68,'E':69.....'Z':90}
    def sam():
        for i in range(65,91):
            yield (chr(i),i)
    print(dict(sam()))
```

Assignment--->04-09-25

#wap to create the following dictionary using generator
#out={'a':97,'b':98,'c':99,'d':100,..'z':122}
#wap to print the num in reverse
5-->5,4,3,2,1
#wap to print n natural numbers
#wap to generate even numbers up to 20
#wap to generate characters from a string--->'PYTHON',exepected: P Y T H O N

Q&A: Generators in Python

Q1. What is a generator in Python?
A generator is a function that uses yield instead of return. It produces values one at a time, pausing after each yield, and resumes from the same point on the next call.

```
def gen():
    yield 1
    yield 2
    yield 3
g = gen()
print(next(g))  # 1
print(next(g))  # 2
print(next(g))  # 3
```

Q2. How is a generator different from a normal function?

- A normal function runs completely once and returns a single value.
- A generator pauses at yield, returns a value, and can continue again until it raises StopIteration.

Q3. What is the difference between yield and return?

- return: exits the function immediately.

- yield: pauses execution, returns a value, and remembers the state for the next call.

Q4. How do we access generator values?
- By calling next(generator_object)
- Or by iterating in a loop (for)
- Or converting into a collection like list(), set(), or tuple().

Q5. What happens when all values are yielded?
The generator raises StopIteration. A for loop handles this automatically.

Q6. Are generators memory efficient? Why?
Yes. Generators don't store all values in memory. They produce values on demand (lazy evaluation). This is useful for large datasets.
Example:

```
# normal list
nums = [x for x in range(1000000)]  # stores all numbers in memory
# generator
nums = (x for x in range(1000000))  # generates on demand
```

Q7. Can a generator be reused after it is exhausted?
No. Once a generator is fully consumed, it cannot be restarted. You need to create a new generator object.

Q8. Can a generator have multiple yield statements?
Yes, it can yield multiple times.

```
def sample():
    yield "Hello"
    yield "Hi"
    yield "Bye"
for msg in sample():
    print(msg)
```

Q9. Can a generator use return along with yield?
Yes, but return ends the generator and raises StopIteration with the return value. The value after return is not part of the yielded sequence.

Q10. What are real-life use cases of generators?
- Reading large files line by line.
- Streaming data from APIs or databases.
- Infinite sequences (like Fibonacci, natural numbers).
- Processing data without storing everything in memory.

Q11. How are generators different from iterators?
- Both are used for iteration.
- An iterator requires a class with __iter__() and __next__() methods.

- A generator is a simpler way to create iterators using yield.

Q12. Can we use generator expressions?
Yes, similar to list comprehension but with parentheses.

```
squares = (x**2 for x in range(5))
print(list(squares))  # [0, 1, 4, 9, 16]
```